

Detecting carbon monoxide plumes from space

A plain-language guide to the two-day machine-learning demonstration

Prepared for the SRON COGNITO post-doc application
Vikrant Tomar · August 2026

The whole idea in three sentences.

A published neural network can spot methane plumes in satellite pictures. It was only ever shown one thing — a small grid of numbers — so it has no idea which gas it is looking at; it recognises *shapes*. The experiment is to point that same detector at carbon monoxide and find out whether shape alone is enough.

Every technical term used here is explained in the glossary in Section 12. Nothing in this document assumes you have worked with neural networks before.

SECTION 1

Why you are doing this at all

SRON is hiring a post-doc for the COGNITO project: build the first global system that spots carbon monoxide plumes in satellite data and works out how much each city and steel plant is emitting. The advertisement asks for two things together — experience interpreting satellite observations, and machine learning.

You have the first comfortably. You have used TROPOMI, GEMS, IASI and OMI in first-author work. You have five years on a reference CO analyser, so you know what a carbon monoxide measurement is actually worth. You use HYSPLIT for transport. Those are strong cards.

The second is where a reader hesitates. Your machine learning is real — you calibrated low-cost sensors against reference analysers with it — but it is *regression on tables of numbers*, and this job is *pattern recognition in pictures*. Those are different corners of the same field. Someone reading your CV cannot tell whether you can do the second.

The point of the demo. Not to make you the strongest machine-learning candidate — someone with deep-learning papers will still look stronger on that axis. It is to remove the reason to set your application aside early, by turning an unknown into a repository someone can open.

What makes it doable in two days

You are not building anything from scratch. Three things already exist and are free:

- **The method.** Schuit and colleagues published it in 2023, and Ilse Aben — one of the two contacts on this vacancy — is a co-author.
- **The training material.** Three thousand small satellite pictures, already sorted by hand into 'plume' and 'no plume', published openly on Zenodo.
- **The carbon monoxide data.** TROPOMI CO is available through Google Earth Engine in a few lines of code.

The advertisement itself says the job is to *apply existing machine learning algorithms*. Not invent. Apply. That is the sentence this whole demonstration leans on.

SECTION 2

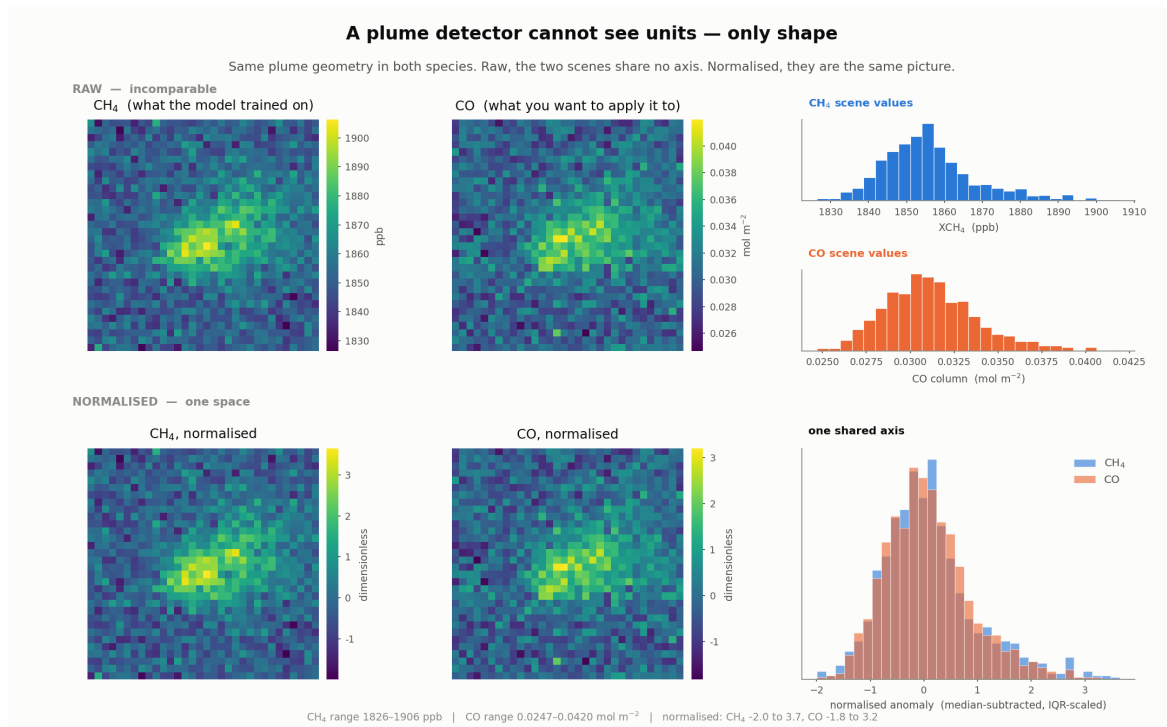
The one idea everything rests on

A detector that has only ever seen normalised numbers cannot know which gas made them.

Here is the thing that makes this work, and it is worth being slow about because everything else follows from it.

The published network was trained on one quantity: the methane column, in parts per billion, roughly 1850. Carbon monoxide is measured as a column density in moles per square metre, roughly 0.030. Different quantity, different unit, five orders of magnitude apart. You obviously cannot feed one to a model trained on the other.

Unless you take the units away first. For each picture separately, subtract the middle value and divide by the spread. Both then become *anomaly maps* — unitless, centred on zero, saying only 'this pixel is high for this scene, that one is low'. You already do exactly this when you remove a baseline before comparing two instrument records.



The same plume geometry rendered as methane and as carbon monoxide. Top row: raw, the two cannot even share an axis.

Bottom row: normalised, they are statistically the same picture.

The numbers, side by side

	Methane	Carbon monoxide
Unit	ppb	mol m ⁻²
Typical background	1854.6	0.0310
Plume peak, raw	1906.2	0.0420
Enhancement above background	+51.6	+0.0110
As a percentage of background	2.8%	35.5%
Plume peak after normalising	3.65	3.20

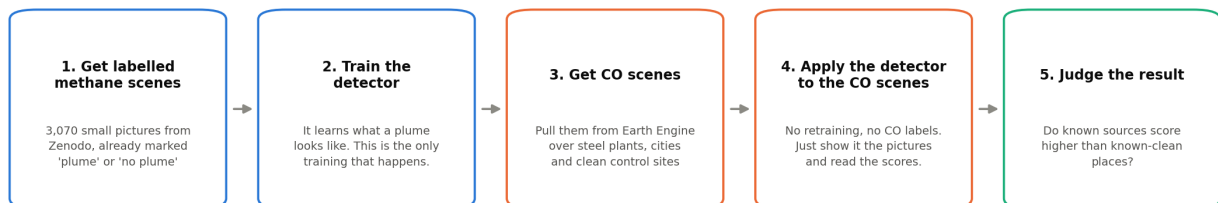
Worth raising before anyone asks you. Look at the percentage row. The CO plume is a far bigger fractional bump than the methane one, so CO looks easier. It is not. CO's background moves around far more, and fires put plume-shaped CO almost everywhere. The difficulty moves from finding the plume to deciding what it is — which is Section 7.

SECTION 3

The whole procedure, in five steps

The experiment, end to end

Five steps. Only one of them involves any training.



The trick: steps 1–2 use methane, steps 3–4 use carbon monoxide. Nothing in between converts one into the other — the detector simply never learns which gas it is looking at.

Step 1 — get the labelled methane pictures

Download three files from Zenodo, about 418 MB. Inside are 828 pictures a human marked 'plume' and 2,242 marked 'no plume', each one a 32 by 32 grid. The licence allows reuse as long as you credit them, which is a condition, not a courtesy.

Step 2 — train the detector

Show it the pictures until it can tell the two groups apart. This is the only training in the entire project. Their published model got 95.6% of the real plumes and was right 94.2% of the time it said 'plume'. You will probably land a little below that, because the exact settings were never published — and saying so is

better than pretending otherwise.

Step 3 — get the carbon monoxide pictures

Pull TROPOMI CO from Google Earth Engine over three kinds of place: seven European steel plants whose emissions were published in 2025, thirteen Indian steel plants and three Indian cities, and three deliberately empty places — the mid-Pacific, the Sahara, and your own site at Nainital.

Step 4 — apply the detector, unchanged

No retraining. No CO labels. Show it the CO pictures and write down the score it gives each one, from 0 to 1.

Step 5 — judge what happened

Did the steel plants score higher than the empty ocean? That is the result. Section 6 explains how you measure that without anyone ever labelling a CO picture.

SECTION 4

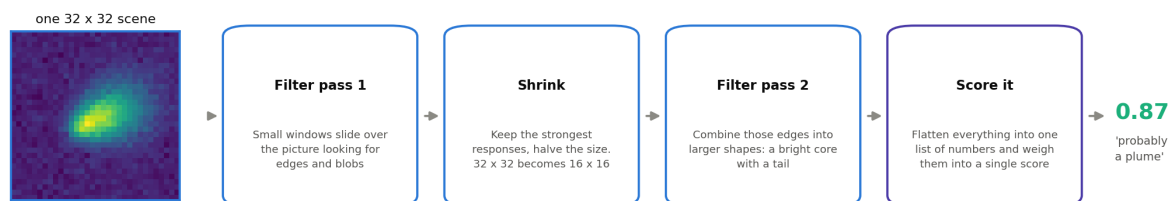
What the detector actually is

A pattern-matcher, not a chemist.

The word 'neural network' does more harm than good here. What this thing does is closer to something you already know: a matched filter. You slide a template across a signal and see where it fits.

What happens inside the detector

A pattern-matcher that looks at shape, then a scorer that turns shape into a number.



The important part: the only thing entering on the left is a grid of numbers. There is no chemistry inside this box, no units, no knowledge of methane. It is a shape detector — which is exactly why it can be pointed at CO.

A convolutional network slides small windows across the picture. Each window is looking for one simple thing — a bright edge, a dark-to-light step, a compact blob. The difference from a matched filter is that nobody writes the templates. They are found automatically during training, by whatever works.

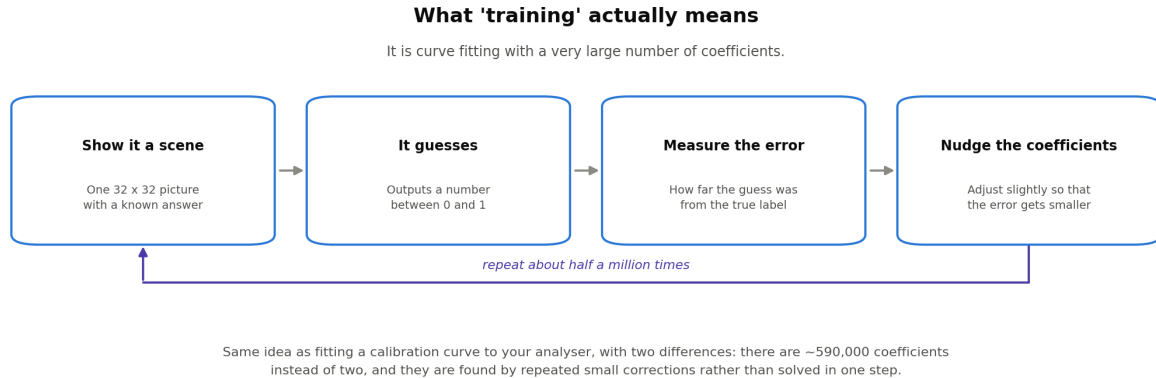
It does this twice. The first pass finds simple features. The picture is then shrunk by half, keeping only the strongest responses, and a second pass combines those simple features into bigger ones — a bright core with a tail stretching away from it, which is what a plume looks like from above. Finally everything is flattened into a list of numbers and weighed into a single score between 0 and 1.

Why this matters for the whole argument. There is no chemistry inside that box. No units, no absorption cross-sections, no knowledge that methane exists. It is a shape detector that happens to have been shown methane pictures. That is the entire reason it is fair to point it at carbon monoxide.

SECTION 5

What 'training' means

Fitting a calibration curve, with 590,000 coefficients instead of two.



The loop is genuinely this simple. Show it a picture whose answer you know. It guesses a number. Measure how wrong the guess was. Nudge every coefficient a little in whichever direction would have made the guess better. Repeat.

There is no insight in there and no understanding. It is a very large number of very small corrections, and the reason it works is that there are enough coefficients and enough examples.

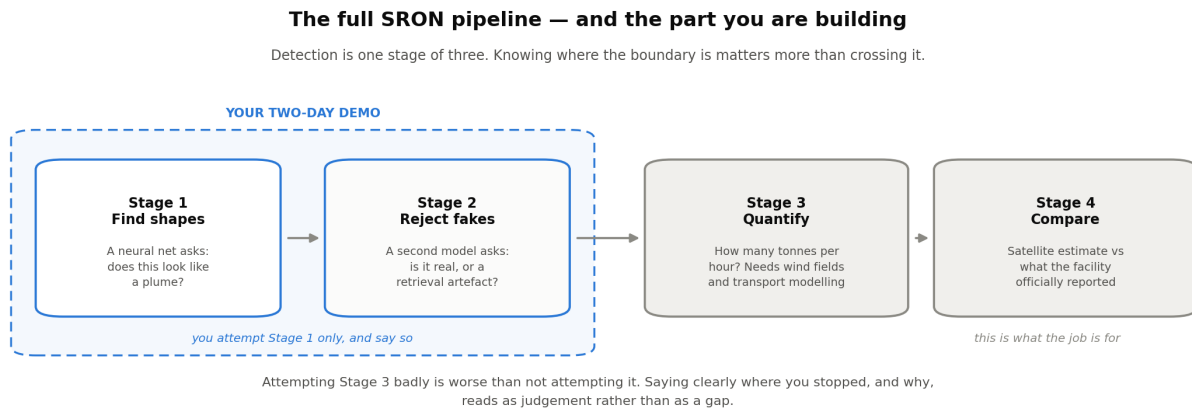
Two habits that come straight from your own work

Keep the test set separate. You would never validate an analyser against the same cylinder you calibrated it with. Same rule. Some pictures are set aside before training and only used at the end to see how well it does on things it has never seen. Score on the training pictures and it looks perfect and means nothing.

Show it every orientation. A plume can point any way the wind blows, and the satellite does not care which way is up. So every picture is rotated four ways and mirrored, turning 3,070 examples into 19,648. This is what the original authors did too.

The one trap in the whole procedure. Split the pictures into train and test *before* you rotate them. Do it the other way round and a rotated copy of a training picture ends up in the test set — the model has effectively already seen the answer, and your score is meaningless. The code in the scaffold does this correctly; do not tidy it away.

Where your two days stop



Finding a plume and measuring a plume are completely different jobs. Finding it needs a picture. Measuring it needs wind fields, a transport model, and a careful estimate of the background — the 2025 steel-plant paper ran eight weather simulations per site just to handle the fact that modelled and observed plumes sit in slightly different places.

You are attempting the first stage only. Two days is enough for that and nowhere near enough for the rest. Attempting the quantification badly would be worse than not attempting it, because it is precisely the thing they are hiring someone to do properly.

How to say it. State the boundary plainly in the README: detection only, no emission rates, and here is why. Naming the limit yourself reads as judgement. Having it pointed out to you reads as a gap.

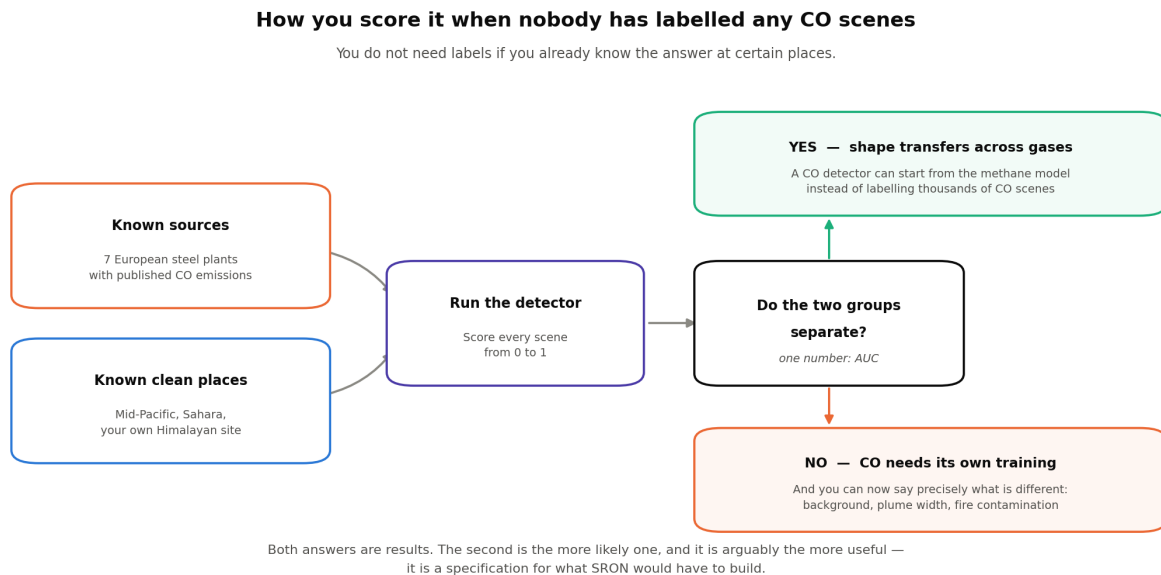
Three other things to leave alone

- **The second stage.** The published method has a follow-up model that filters out false plumes caused by the retrieval itself. It needs all thirteen data channels and 41 derived quantities. Understanding why it exists is worth more than a rushed version.
- **The raw Level-2 data.** Regridding it properly is a day on its own. Use the ready-made gridded product from Earth Engine and say clearly that you did.
- **Polishing the code.** They are not hiring a software engineer. A clear README beats tidy abstractions every time.

How you score it with no labelled CO data

The fair objection: nobody has ever marked up CO pictures, so what do you compare your answers against?

You do not need labels if you already know the answer somewhere. Pick places where the truth is not in doubt, and see whether the detector can tell them apart.



Seven European steel plants had their CO emissions published in 2025, so a large source is definitely there. The middle of the Pacific, the Saharan interior and your own high-altitude Himalayan site definitely have no large industrial CO source. If the scores separate those two groups, shape transferred. If they overlap, it did not.

That separation is a single number — the area under the ROC curve, or AUC. One means perfect separation, 0.5 means the detector is guessing.

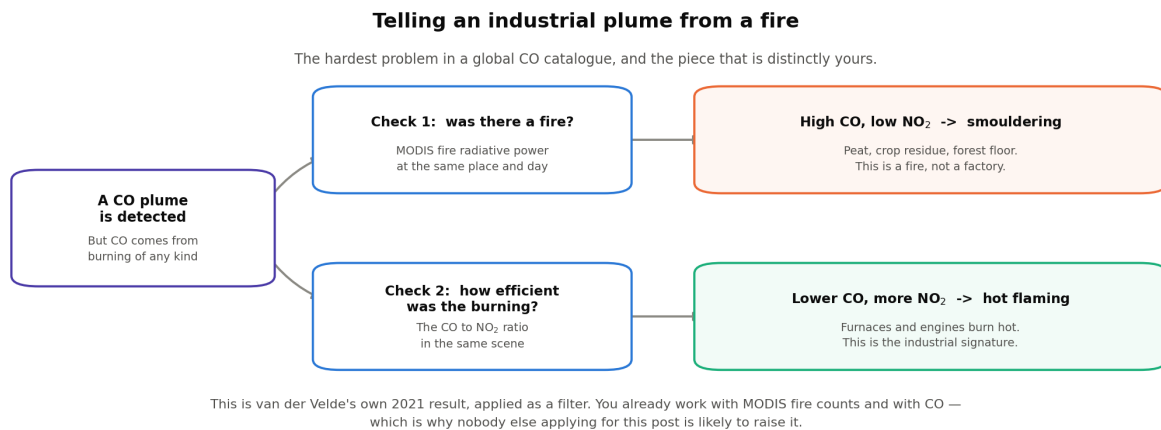
Then look at the pictures

A hundred scenes is twenty minutes of looking, and it is not optional. A number can separate two groups for the wrong reason — the model might be firing on the edge of the satellite swath, or on a cloud boundary, and still score well because those happen more over land than over ocean. Never report a separation you have not seen with your own eyes.

Fires — the hard part, and your part

Carbon monoxide comes from burning. All burning.

A steel furnace makes CO. So does a crop fire, a forest fire, a smouldering peatland, and every vehicle in a city. A global catalogue of CO plumes built from 2018 onward would be full of fires unless somebody separates them out, and over the Indo-Gangetic Plain in burning season that is most of what you would find.



There are two independent checks, and they come from different places.

Was there a fire? MODIS sees fire radiative power directly. If it registers a fire in the same place on the same day, that is a strong hint — and you already work with MODIS fire counts.

How hot was the burning? This is the clever one, and it is Ivar van der Velde's own 2021 result. Efficient, hot, flaming combustion makes relatively more NO₂; cool smouldering makes relatively more CO. So the ratio of CO to NO₂ in the same scene tells you what kind of fire you are looking at. Furnaces and engines burn hot. Peat and crop residue smoulder.

This is the piece that is distinctly yours. It is his own published method, applied to the contamination problem his own project will hit. You already work with CO, with NO₂, and with MODIS fire counts over exactly the region where this is worst. It is unlikely anyone else applying will raise it.

One deliberate omission in the code

The threshold that separates 'smouldering' from 'flaming' is left unset. There is no universal value — it depends on the units, the fuel, the season and how long NO₂ survives in the air. The code plots the ratio for fire scenes against non-fire scenes so you can read the threshold off your own data. Shipping a plausible-looking constant you had not checked, to the man who published the method, would be worse than shipping none.

SECTION 9

What each file in the folder does

File	What it does	When you run it
selftest.py	Invents fake plumes and checks the whole machine works end to end. No downloads needed.	First, before anything
get_training_data.py	Downloads the three labelled methane files from Zenodo, about 418 MB.	Day 0, leave it running
dataset.py	Reads the files, removes the units, and makes the four rotations and two mirrors of each picture.	Called by the others
model.py	The detector itself — two filter passes and a scorer, about 590,000 coefficients.	Called by the others
train.py	Runs the training loop and reports how it did against the published 95.6% and 94.2%.	Day 1 morning
fetch_co_tiles.py	Pulls TROPOMI CO, NO ₂ and MODIS fire power from Earth Engine around each target.	Day 1 afternoon
predict_co.py	The actual experiment. Scores the CO pictures, flags likely fires, and draws the contact sheet.	Day 2 morning
facilities.csv	The 26 places: 7 European steel plants, 13 Indian, 3 cities, 3 empty controls.	Edit before fetching
PROVENANCE.md	Which parts come from the papers and which are judgement calls you would have to defend.	Read before publishing

On authorship. Every line of that Python was written for you as a starting point — none of it is the original authors' code, which was never released. It becomes your work once you have read it, changed what you disagree with, and can defend the choices in PROVENANCE.md. Put a line in the README saying the scaffold was AI-assisted and what you did with it. That is ordinary practice now, and much safer than being asked later.

The two-day timetable

When	What	You are done when
Day 0 one hour, tonight	Install everything. Run selftest.py . Start the Zenodo download and leave it. Register for Earth Engine.	It prints SELFTEST PASSED
Day 1 morning	Train on the methane pictures. Mostly waiting.	Recall and precision both somewhere around 0.85 to 0.95
Day 1 afternoon	Fetch CO pictures. Start with the seven European plants only — they are the control. Add the Indian sites once those work.	You have roughly 100 or more usable scenes
Day 2 morning	Run the detector on the CO pictures. Look at the contact sheet. Check the fire flags.	You can say in one sentence what happened
Day 2 afternoon	Write the README: the question, what you did, what happened, what it means, the caveats. Push to GitHub.	The link works from a private browser window

If something goes wrong

- **Download or training fails** — use the third Zenodo file instead, which is smaller and faster. A smaller demonstration that works beats a bigger one that does not.
- **Earth Engine is slow or hits a quota** — cut to five facilities and twenty overpasses each. A hundred pictures is enough. Do not let data collection eat Day 2.
- **You run out of time** — ship what works with an honest README. An unfinished repository with a clear account of where it stopped is still far better than no repository.

What to say, and what not to

One sentence in the motivation letter. Not a paragraph — the repository does the talking:

To test whether the detection approach transfers across species, I reproduced the neural-network stage of Schuit et al. (2023) on your published training scenes and applied it to TROPOMI CO columns over European and Indian steel plants. [One clause on what you found.]

Things you can say honestly

- You reproduced the *approach*, from the paper's description.
- You applied it to CO without retraining, and here is what happened.
- You know why the method needs a second stage, and you did not attempt it.
- You know what the gridded product costs you compared with the raw data.
- You built a fire filter using their own combustion-efficiency result.

Things that would be untrue

- That you reproduced *their model*. You reproduced their approach — the exact settings were never published.
- That you quantified emissions. You detected plumes. Different job.
- That you used Level-2 data. You used the harmonised gridded product.
- That the fire threshold is calibrated, unless you have actually calibrated it.

Why the caveats are the strongest part. SRON built the TROPOMI carbon monoxide retrieval. Ilse Aben co-wrote the paper you are reproducing. These are the last people on earth to bluff in front of — every limitation above is one they would spot in half a minute. Naming them first turns each one from a weakness into evidence that you understand the method. It is the same instinct that makes your instrument background worth something: knowing what a measurement can and cannot support.

Glossary, in your language

Term	What it means here
Neural network / CNN	A pattern-matcher built from sliding filters. Closer to a matched filter than to anything that thinks.
Training	Fitting coefficients by repeated small corrections, rather than solving in one step. Same purpose as fitting a calibration curve.
Label	The known right answer for one example — here, a human saying 'this picture has a plume'.
Scene / tile	One 32 by 32 patch of the satellite grid, about 220 km across.
Normalisation	Removing the units by subtracting the middle value and dividing by the spread. The same move as taking anomalies from a baseline.
Augmentation	Making extra training examples by rotating and mirroring the ones you have, since a plume has no preferred direction.
Train / test split	Holding some examples back so you can check performance on data the model never saw. Like keeping a validation cylinder separate from the calibration cylinder.
Overfitting	Learning the noise in your examples rather than the signal. A calibration curve forced through every scattered point.
Recall	Of all the real plumes, what fraction did it find? Their model: 95.6%.
Precision	Of everything it called a plume, what fraction really was? Their model: 94.2%.
AUC	How well the scores separate two groups. 1.0 is perfect, 0.5 is guessing.
Class imbalance	Far more negatives than positives, so a model that says 'no plume' every time scores 73% and is useless. Handled by weighting the errors.
Artefact	A plume-shaped feature caused by the retrieval rather than the atmosphere. The reason the published method needs a second stage.
Level 2 / Level 3	L2 is the retrieval on the satellite's own footprints. L3 is regridded onto a regular map. You are using L3.

The three papers to read

All three are open access. Read them before writing the letter — the letter refers to them specifically, and being caught referring to a paper you have not read would undo everything else.

1. The method you are reproducing

Schuit, B. J., Maasakkers, J. D., Bijl, P., *et al.*, including **Aben, I.** (2023). Automated detection and monitoring of methane super-emitters using satellite data. *Atmospheric Chemistry and Physics* **23**, 9071–9098.

Read sections 2 and 3 for the network and the two-stage design. Training data: doi.org/10.5281/zenodo.13903869, CC-BY 4.0 — credit it.

2. The carbon monoxide precursor that COGNITO scales up

Leguijt, G., Maasakkers, J. D., Denier van der Gon, H. A. C., Segers, A. J., Borsdorff, T., **van der Velde, I. R.**, and **Aben, I.** (2025). Comparing space-based to reported carbon monoxide emission estimates for Europe's iron and steel plants. *Atmospheric Chemistry and Physics* **25**, 555–574.

Twenty-one plants, 2019. Satellite estimates averaged 17% below what the plants reported, and two plants reported cuts the satellite could not see. That last finding is what the whole programme is politically for.

3. Your hiring manager's own work on fires

van der Velde, I. R., van der Werf, G. R., Houweling, S., Eskes, H. J., Veefkind, J. P., Borsdorff, T., and **Aben, I.** (2021). Biomass burning combustion efficiency observed from space using measurements of CO and NO₂ by TROPOMI. *Atmospheric Chemistry and Physics* **21**, 597–616.

The source of the fire filter in Section 8. He is first author on this, on the Nature paper about the Australian fires, and on the 2024 southern Africa small-fires paper. Fires and CO are his line of work, and they are close to yours.

A closing thought

The demonstration is not the strongest thing in your application, and it is not meant to be. What carries this application is the combination nobody else has: you have measured carbon monoxide with your own hands for five years, you work on biomass burning in the region where CO contamination is worst, and you can speak to Indian steel — the second-largest producer in the world, and the place where a global catalogue will be hardest to trust because no facility there reports the way European ones do.

Two days of code removes the one thing standing in front of all of that.